LESSON 25

同一个 盒子



引用 &

学完以后，函数能直接改外面的变量，还能一次交回好几个结果。

addOne 加了 1，外面的 x 怎么没变？

add_one.cpp（只阅读）

```
 4 void addOne(int x) {
 5     x = x + 1;
 6 }
 8 int main() {
 9     int x = 0;
10     cin >> x;
11     addOne(x);
12     cout << x << endl;
```

开头的 void 表示这个函数只做事，不用 return 交回值。

输入 5

5

第 18 课见过：函数拿到的是一份副本，改的是函数里自己的 x。

想让函数真的改到外面的 x，要换一种写法。

加一个 &，改的就是外面那个

add_one.cpp

```cpp
void addOne(int &x) {
    x = x + 1;
}
int main() {
    int x = 0;
    cin >> x;
    addOne(x);
    cout << x << endl;
```

输入 5

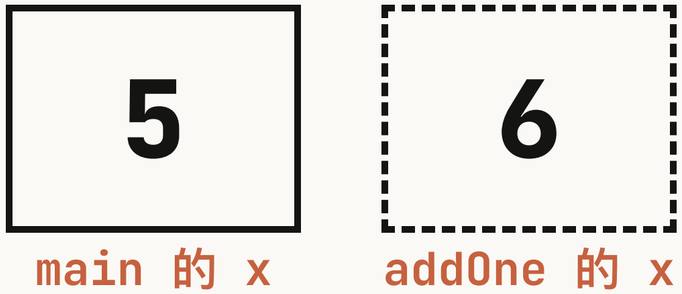
6

int &x 叫**引用**：不复制，函数里的 x 就是外面那个 x，换了个名字而已。

& 只写在函数头里。调用时照常写 addOne(x)。

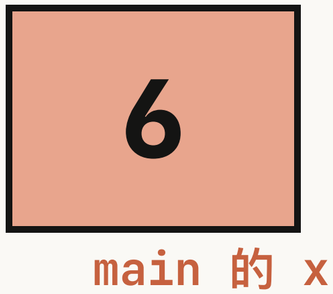
副本和引用，差在哪

`int x`：两个盒子



复制一份过去，改的是右边那个，函数结束就扔掉了。

`int &x`：一个盒子，两个名字



只有一个盒子，函数里改了，外面马上看得到。

小测：x 一开始是 3，调用两次后输出几？

```
void twice(int &a) { a = a * 2; }
int x = 3; twice(x); twice(x); cout << x;
```

1 3

2 6

3 12

4 报错

答对了！3 变 6，6 再变 12。

swapTwo：真的把外面两个数换过来

swap.cpp

```cpp
void swapTwo(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

输入 5 3

```
3 5
```

换的办法和第 21 课一样，先用 temp 存一下。

swap.cpp

```cpp
    swapTwo(a, b);
    cout << a << " " << b << endl;
```

两个参数都有 &，两个都能改到。

猜一猜：函数头里的两个 & 都去掉

swap.cpp（只阅读）

```
4 void swapTwo(int a, int b) {
```

屏幕上

5 3

别的都不变，还是输入 5 3，屏幕上是什么？

没换。函数换的是两份副本，外面的 a、b 一点没动。
想改外面的变量，函数头里就要写 &。

动手改一改

从 swap 开始。什么时候才需要换?

小的在前

读入两个整数，用 swapTwo 让它们从小到大输出，中间空一格。本来就是小的在前，就不用换。

输入 5 3

```
3 5
```

输入 3 5

```
3 5
```

Dev-C++: 打开 starter/03_order2.cpp

一次 return 只交回一个值，商和余数怎么办?

divide.cpp

```cpp
void divide(int a, int b, int &q, int &r) {
    q = a / b;
    r = a % b;
}
```

divide.cpp

```cpp
    int q = 0, r = 0;
    divide(a, b, q, r);
    cout << q << " " << r << endl;
```

输入 17 5

3 2

a、b 只是拿来用，不用 &。

q、r 是要填结果的，加 &。

第 19 课的结构体也能把两个数装在一起交回，这节课用引用。

动手改一改

从 divide 开始。哪几个参数要加 &?

最小和最大一起交回

读入三个整数。写一个函数 minMax(a, b, c, lo, hi)，把最小的放进 lo，最大的放进 hi。输出 lo 和 hi，中间空一格。

输入 3 1 2

```
1 3
```

输入 5 5 5

```
5 5
```

Dev-C++：打开 starter/05_min_max.cpp

把 vector 交给函数，也用 &

add_all.cpp

```
 5 void addAll(vector<int> &v, int k) {
 6     int n = v.size();
 7     for (int i = 0; i < n; i++) {
 8         v[i] = v[i] + k;
 9     }
10 }
```

add_all.cpp

```
21     addAll(v, 10);
```

输入 3 和 1 2 3

11 12 13

有 &，函数改的就是外面的 v。vector 很长的时候，也省得复制一整份。

这个程序先输入个数 n，再输入 n 个整数。和第 23 课读到 0 才停不一样。

小测：写成 vector<int> v，不加 &，输出是？

还是输入 3 和 1 2 3，别的都不变。

1 11 12 13

2 1 2 3

3 报错

4 10 10 10

答对了！加的是副本，外面的 v 没变。

读报错：addOne(5) 不行

add_one.cpp（只阅读）

```
9       addOne(5);
```

抓住一个词：**reference**，引用。

```
9:12: error: cannot bind non-const lvalue reference of type 'int&' …
```

addOne 要把交给它的东西加 1。5 是一个数，不是盒子，没法改。
参数写了 &，调用时就要给一个变量。

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

找错：哪一行让 b 没换到？

swap.cpp（只阅读）

```
1 void swapTwo(int &a, int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }
```

输入 5 3，应该是 3 5，
屏幕上却是 3 3。
点你认为错了的那一行。

点代码里的一行

自己挑战

三个数排排队

写一个函数 sortThree(a, b, c)，把外面的三个变量从小到大排好。读入三个整数（-1000 到 1000），调用它，再输出 a b c，中间空一格。

输入 3 1 2，输出

1 2 3

输入 5 5 1，输出

1 5 5

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　starter 里已经有 swapTwo。sortThree 的三个参数都要加 &。

提示 2　像冒泡一样，相邻两个比：a 比 b 大就换，b 比 c 大就换。

提示 3　只比一轮够不够？用 3 2 1 试一试，不够就把 a 和 b 再比一次。

角色升级

写一个函数，一调用就同时改好几样东西。主题你自己定。

先做到

函数用两个引用参数，调用一次，两样都变。

选你喜欢的

角色升级、存钱罐、宠物喂食、闯关加分，都可以。

还想更厉害

调用好几次，看看数字一次次怎么变。

没有标准答案。写之前说清想要的结果，再试试一开始就是 `0`。

和同桌换着玩

1

换着运行

运行同桌的程序，换几个数试试，再试试 `0`

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：哪个参数要 `&`，哪个不用，为什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **`int x` 和 `int &x` 当参数，差在哪?**

`int x` 复制一份；`int &x` 就是外面那个变量，改了外面看得到。

Q2 **函数要交回两个结果，怎么办?**

多写两个引用参数，在函数里把结果放进去。

Q3 **参数是 `int &x`，能写 `addOne(5)` 吗?**

不能，要给一个变量，5 没法被改。

速查卡

`void f(int &x)`	x 就是外面那个变量
`f(x);`	调用时不写 &，要给变量
`f(int a, int &q)`	拿来用的不加 &，要填结果的加 &
`f(vector<int> &v)`	改得到外面的 v，也不复制一整份

这些后面再学

指针

& 还有另一个意思：写在变量前面，是“它住在哪里”

const &

只想省掉复制、又不想被改到时用

今天学会引用：加一个 &，函数里的名字就是外面那个变量。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：指针入门